The Chinese University of Hong Kong, Shenzhen
School of Data Science

DDA5001 · Homework 3
Due: 23:59, Nov 13.

Instructions:

- Homework problems must be clearly answered to receive full credit. Complete sentences that establish a clear logical progress are highly recommended.
- Submit one **PDF** with your answers and your **.py** code through Blackboard, combined in **last name-first name-hw3.zip**; e.g., **San-Zhang-hw3.zip**.
- You may submit your homework multiple times before the deadline. Only your **latest submission will be graded**.
- The homework must be written in English.
- Late submission will not be graded.
- Discussion with peers is allowed, but do not copy another student's solution or a ready-made solution. Coding-agent assistance is **required for model construction in Problem 3(b)**. Disclose that collaboration as requested in Problem 3(d). Check any generated code, and write the mathematical derivations and explanations in your own words. You are responsible for understanding and verifying all submitted work.

Problem 1 (30pts). Regularization selected by five-fold validation

Suppose that the underlying model is

$$y = x^2 + \varepsilon.$$

The quadratic curve $y = x^2$ is the clean target, while ε is observation (stochastic) noise. We collect $n = 10$ training pairs $\{(x_i, y_i)\}_{i=1}^{n}$, stored in `data/training.csv`. Figure 1 shows these observations and the clean target: the observations need not lie on the curve because they contain noise. A separate set of 10 test pairs is supplied in `data/test.csv` for final evaluation. In the released Problem 1 code package, both paths are relative to `p1.py`.

Point IDs. The ten training observations are numbered **0–9 in CSV row order**, excluding the header. Thus point ID 0 is the first data row, point ID 1 is the second, and so on; ID r corresponds to (x_{r+1}, y_{r+1}). The arrows in the figure identify these points. **The validation folds in part (a) use these same IDs**, not positions after sorting by x or renumbering a subset.

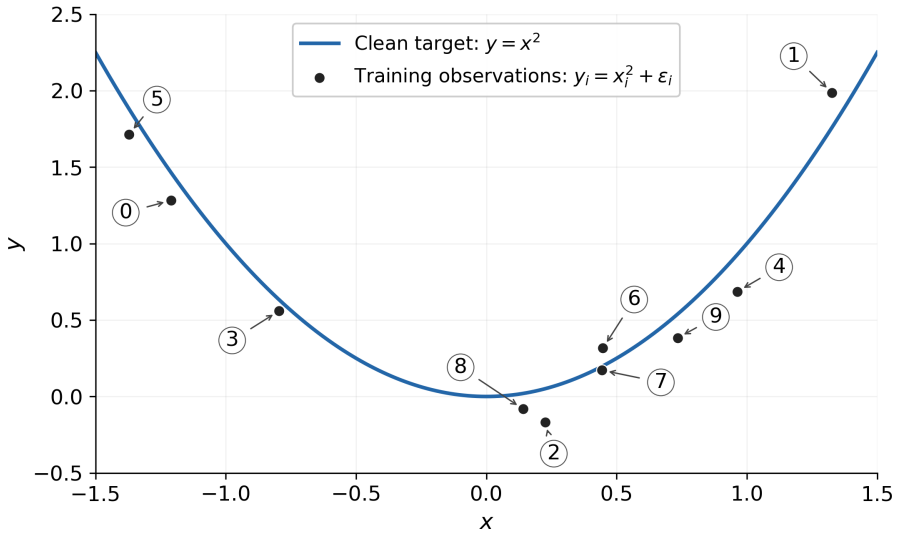


Figure 1: The original ten noisy training observations and the clean quadratic target. Point IDs 0–9 follow the rows of `training.csv`.

Polynomial model and least squares. We fit a degree-8 polynomial with parameter vector $\boldsymbol{\theta} = [\theta_0, \ldots, \theta_8]^\top \in \mathbb{R}^9$:

$$f_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_8 x^8.$$

This model is nonlinear in the input x, but linear in the unknown coefficients $\boldsymbol{\theta}$. For the full training set, define

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^8 \\ 1 & x_2 & x_2^2 & \cdots & x_2^8 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^8 \end{bmatrix} \in \mathbb{R}^{n \times 9}, \qquad \boldsymbol{y} = [y_1, \ldots, y_n]^\top \in \mathbb{R}^n.$$

The predictions on these inputs are $\boldsymbol{X\theta}$. for fitting $\boldsymbol{y}$. A fit on a subset uses only the corresponding rows of $\boldsymbol{X}$ and $\boldsymbol{y}$. In the training objectives below, m denotes the number of observations used for that fit: $m = 10$ for a full-data fit and $m = 8$ for each cross-validation training fit.

The unregularized least-squares baseline minimizes the discrepancy between predictions and observations:

$$\widehat{\boldsymbol{\theta}}_0 = \operatorname*{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^9} \frac{1}{m} \|\boldsymbol{X\theta} - \boldsymbol{y}\|_2^2.$$

The degree-8 model has more flexibility than the clean quadratic target and may overfit noise as well as normal-fitting the underlying trend. This can produce a small training error but a large error on unseen data (overfitting). Greater degree alone does not necessarily prove overfitting; the fitted curves and errors provide evidence in this experiment.

Regularization and validation. To mitigate possible overfitting, add an ℓ_2-regularizer:

$$\widehat{\boldsymbol{\theta}}_\lambda = \operatorname*{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^9} \frac{1}{m} \|\boldsymbol{X\theta} - \boldsymbol{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2.$$

The remaining question is how to choose λ. Five-fold cross-validation estimates prediction error using training observations held out from each fit: fit on eight points, validate on the other two, and rotate the held-out pair so every training point is validated once. Average the five validation errors, select λ, and refit on all ten training points. The independent test set is used only for the final comparison, **not for selecting** λ. Every reported error in this problem is **mean squared error (MSE)**, defined by $\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(f_{\boldsymbol{\theta}}(x_i) - y_i)^2$ for the N observations being evaluated. Note that this MSE error include all training error, validation error, or test error as special cases, depending on what data are used for calculating this error.

(a) **(18pts)** Fit the regularized model **(6pts)** and use **five-fold cross-validation (8pts)** to select λ from

$$\{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 0.3, 0.5, 0.8, 1, 2, 5, 10, 15, 20, 50, 100\}.$$

Use the following fixed validation folds, expressed using the point IDs in Figure 1:

$$\{6, 0\}, \quad \{3, 9\}, \quad \{2, 1\}, \quad \{4, 8\}, \quad \{5, 7\}.$$

For each fold, fit on the other eight rows and compute MSE on the two held-out rows. Average the five MSEs. Report the selected λ and plot average validation MSE against λ using a log-scale horizontal axis **(4pts)**. Break an exact tie by choosing the smaller λ.

(b) **(12pts)** Refit the selected model on all 10 training rows. In **one plot**, compare its curve with the unregularized least-squares fit ($\lambda = 0$), the training points, and the clean target x^2 **(4pts)**. Use $x \in [-1.5, 1.5]$ and $y \in [-0.5, 2.5]$. Report training and test MSE for both fits **(4pts)**. In two or three sentences, explain the observed tradeoff and why λ was selected using validation rather than test error **(4pts)**.

Note: Use the supplied `p1.py` framework and complete only its two marked `TODO` parts: final regularized fitting and cross validation scoring/lambda selection. Data loading, MSE, CV model fitting, the unregularized baseline, and plotting are provided.

Problem 2 (20pts). PCA face reconstruction

We apply PCA to the ORL face dataset, which contains **400 images: 40 people with 10 images per person**. Figure 2 shows some examples.



Figure 2: Example images from the ORL face dataset (a subset of the 400 images).

Use the images in `data/p2/ORL_faces` at their original 112×92 resolution. Order them by subject 1–40 and image 1–10 within each subject, flatten each image into a column vector $\boldsymbol{x}_i \in \mathbb{R}^{10304}$, and divide pixel values by 255. Thus each coordinate of $\boldsymbol{x}_i$ is one pixel intensity, and each column of

$$\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{400}] \in \mathbb{R}^{10304 \times 400}$$

represents one face. Before applying PCA, subtract the mean face from every image:

$$\boldsymbol{\mu} = \frac{1}{400} \sum_{i=1}^{400} \boldsymbol{x}_i, \qquad \boldsymbol{X}_c = \boldsymbol{X} - \boldsymbol{\mu}\mathbf{1}^\top.$$

Basis faces and coordinates. PCA represents the centered faces in a k-dimensional subspace by solving

$$\min_{\boldsymbol{A}_k^\top \boldsymbol{A}_k = \mathbf{I},\ \boldsymbol{\Theta}_k} \|\boldsymbol{X}_c - \boldsymbol{A}_k \boldsymbol{\Theta}_k\|_F^2.$$

Here $\boldsymbol{A}_k = [\boldsymbol{a}_1, \ldots, \boldsymbol{a}_k] \in \mathbb{R}^{10304 \times k}$ is the **basis matrix**. Its columns are orthonormal patterns shared by all the faces. Each $\boldsymbol{a}_j$ can be reshaped into a 112×92 image and is called an **eigenface**. The matrix $\boldsymbol{\Theta}_k = [\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_{400}] \in \mathbb{R}^{k \times 400}$ contains the **low-dimensional coordinates**: column $\boldsymbol{\theta}_i$ gives the k weights used to represent face i. The objective minimizes the total squared reconstruction error across all centered images.

Computing and reconstructing. Compute the **thin SVD** $\boldsymbol{X}_c = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$, with singular values in descending order. An optimal basis $\boldsymbol{A}_k$ consists of the first k columns of $\boldsymbol{U}$. Project onto this basis to obtain the coordinates, and add the mean face back after reconstruction:

$$\boldsymbol{\Theta}_k = \boldsymbol{A}_k^\top \boldsymbol{X}_c, \qquad \widehat{\boldsymbol{X}}_k = \boldsymbol{\mu}\mathbf{1}^\top + \boldsymbol{A}_k \boldsymbol{\Theta}_k.$$

For an individual image, this means

$$\widehat{\boldsymbol{x}}_i = \boldsymbol{\mu} + \boldsymbol{A}_k \boldsymbol{\theta}_i = \boldsymbol{\mu} + \sum_{j=1}^{k} \boldsymbol{a}_j \boldsymbol{\theta}_i[j],$$

where $\boldsymbol{\theta}_i[j]$ is the j-th entry of $\boldsymbol{\theta}_i$. Thus a reconstructed face is the **mean face plus a weighted linear combination of the eigenfaces**; the coordinates specify how much of each eigenface is used.

The choice of k controls how many basis patterns and coordinates are retained. A small k gives a more compact representation but may lose image details; a larger k can preserve more variation at the cost of storing more coordinates. The experiment below compares $k = 20$ and $k = 100$ through both reconstructed images and numerical error. You do not need to form a 10304×10304 covariance matrix.

(a) **(8pts)** Display the **first 16 eigenfaces** as one 4×4 grid, in descending singular-value order. The SVD computation carries **(4pts)** and the correctly ordered display carries **(4pts)**.

(b) **(12pts)** Select image 1 of subjects 1–5 (zero-based columns $0, 10, 20, 30, 40$). Display one 5×3 grid: original image, reconstruction with $k = 20$, and reconstruction with $k = 100$ **(6pts)**. Report the MSE $\|\widehat{\boldsymbol{X}}_k - \boldsymbol{X}\|_F^2/(10304 \cdot 400)$ for the **whole dataset** at these two values of k **(3pts)**. Explain in two or three sentences what changes as k increases **(3pts)**.

Note: Use the supplied `p2.py` framework and complete only its three marked `TODO` parts: constructing the data matrix, computing the PCA basis, and projecting/reconstructing the images. Image loading and normalization, MSE, and plotting are provided.

Problem 3 (50pts). Neural network modeling with a coding agent

Note: The use of a coding agent is required in part (b) of this question.

Multi-class logistic regression maps an input directly to class scores. A neural network can first transform the input through a hidden layer and then produce those scores. Does adding a layer alone change what the model can represent? What is the role of a nonlinear activation? In this problem, you will build three models and use a supplied training framework to investigate these questions.

Data and provided framework. We use MNIST handwritten-digit images. Each 28×28 image is represented by $\boldsymbol{x} \in \mathbb{R}^{784}$, and its label is one of the ten digits $0, \ldots, 9$. The framework provides data loading and preprocessing, a single fixed training, validation, and test split, training and validation-based checkpoint selection, final testing, and plotting. It does not use cross-validation, while using the fixed validation set for computing the validation error. We will use the optimizer "SGD with Momentum" for training, which applies the "GD with momentum" idea to minibatches rather all training data points at each iteration; its implementation is supplied in the framework; you do not need to implement it by yourself, and you will study it in later lectures. **Your implementation task is only to construct the three models below, not to write the training algorithm or plotting code.**

Three models. Each model returns ten raw class scores, called **logits**, collected in $\boldsymbol{t}$. The supplied cross-entropy loss uses these logits directly, so do not put a Softmax layer at the model output. Every linear layer includes a bias:

$$\begin{aligned}
&\text{A (linear baseline):} && \boldsymbol{t} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}, \\
&\text{B (two linear layers):} && \boldsymbol{t} = \boldsymbol{W}_2(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2, \\
&\text{C (with ReLU):} && \boldsymbol{t} = \boldsymbol{W}_2\,\mathrm{ReLU}(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2.
\end{aligned}$$

Here $\mathrm{ReLU}(z) = \max\{z, 0\}$ is applied to each coordinate. Both B and C use hidden width 64. They have the **same layer dimensions, depth, and number of trainable parameters**; the only architectural difference is the ReLU in C. This is the controlled comparison for studying nonlinearity. A is a single-layer reference model (just the normal multi-class logistic regression).

In this question, comparing A and B shows how depth alone (without nonlinear activation function) will affect the performance. Comparing B and C further shows how the non-linearity, i.e., ReLU activation, affects the performance of a neural network.

(a) **(8pts) Predict before running.** In a few sentences, predict whether stacking the two linear layers in B changes the kind of input-to-logit functions available in A **(4pts)**. Explain what you expect adding ReLU to change, and why comparing B with C helps isolate its effect **(4pts)**. Give reasons; you do not need to predict exact accuracies.

(b) **(14pts) Construct the three models with a coding agent.** Work with an agent to complete the A, B, and C model definitions in the provided PyTorch framework **(9pts)**; each correct model carries **(3pts)**. Briefly explain how your layer definitions correspond to the formulas above, including the role and placement of ReLU and the input/output dimensions **(5pts)**. Use the provided training and evaluation code without changing its settings. You do not need to implement an optimizer, backpropagation, or a loss function. Complete only the three model-building `TODO` functions in `p3.py`; all other code is provided.

(c) **(18pts) Run and interpret the comparison.** Run the given framework for the three models; its plotting and evaluation code is supplied.

- Submit one two-panel figure showing training and validation cross-entropy versus epochs, with A, B, and C in each panel **(6pts)**.
- Submit one three-row table showing each model's validation-selected epoch, validation cross-entropy, and corresponding test cross-entropy and test accuracy **(4pts)**. The framework selects checkpoints using validation error, not test error. Identify which model is preferred by validation error.
- Compare A with B, and then use the **matched B–C pair** to discuss the observed effect of nonlinearity **(4pts)**. Support your discussion with your curves or table, and revisit your predictions in (a). Do not attribute an A–C difference solely to activation, since their parameter counts differ.
- Discuss the training–validation differences and why a validation-selected checkpoint may be preferable to the last epoch **(4pts)**. Describe what your run shows; neither a prescribed model ranking nor a prescribed overfitting pattern is required.

(d) **(10pts) Record the collaboration in part (b).** Submit at most one page describing one actual model-construction interaction with your agent. You may consider to include: the question or concern you raised; your actual prompt and the relevant reply excerpt, identifying the tool/model if shown; how you checked the proposed model and any evidence-based follow-up; and your final decision to keep or revise it, with a reason. Checks can use the layer definitions, dimensions, or outputs from the provided framework. A correct first implementation can receive full credit: do not invent a bug or submit a full conversation dump. Retain the original interaction for possible review. No separate interaction record is required for simply running the framework in (c).

Note: Only part (b) requires coding: complete the three A/B/C model-construction `TODO` functions in `p3.py` with the help of a coding agent. Data loading and splitting, optimization, training, validation, testing, and plotting are all provided.